

SNOWFLAKE

CDC & SCD Complete Pipeline

AWS S3 → Bronze → Silver → Gold

Hash-Based CDC · Stream-Based CDC · SCD Types 0/1/2/3/4/6

Point-in-Time FK Resolution · CDC Event Log · Full Task DAG

CHAPTER 1

What Is CDC (Change Data Capture)?

CDC is the technique of identifying and capturing only the rows that CHANGED in a source system since the last extract — rather than reloading everything each time. This is the foundation of incremental pipelines.

1.1 The Problem CDC Solves

Without CDC, you reload the entire source table every day. With 100M rows and daily changes of only 0.1%, that means loading 100M rows to get 100K actual changes — 99.9% wasted I/O. CDC identifies only the 100K changed rows and processes those.

Approach	Load Volume	Processing	Latency	Complexity
Full reload	100% of rows every run	Simple — overwrite target	Hours	Very low
Timestamp CDC	Rows where <code>updated_at > last_run</code>	Medium — needs watermark	Minutes	Low
Hash CDC	Rows where <code>hash(cols)</code> changed	High — compute + compare hashes	Minutes	Medium
Stream CDC (Snowflake)	Only rows changed since stream offset	Low — Snowflake native	Minutes	Low
Log CDC (Debezium)	Real DB transaction log	Near-zero overhead on source	Seconds	High

1.2 CDC Patterns Used in This Project

Pattern A — Hash-Based CDC

At load time, compute MD5(all business columns) and store as `_SRC_ROW_HASH`. Compare against the last-known hash in `CDC_STATE`. If hashes differ → row changed. This works even when the source system has no `updated_at` column.

Pattern A: Hash-Based CDC

```
-- Computing row hash at COPY INTO time
COPY INTO BRONZE_DB.RAW.RAW_CUSTOMERS(..., _SRC_ROW_HASH)
FROM (
  SELECT ...,
    MD5(CONCAT_WS('|',      -- pipe-separated for collision safety
      $1,  -- customer_id
      $2,  -- customer_name
      $3,  -- email
      $4,  -- phone
      $5,  -- segment
      $6,$7,$8,$9,$10,$11,$12,$13,$14,$15
    )) AS _SRC_ROW_HASH
  FROM @stage
)

-- Later: compare current hash vs CDC_STATE.LAST_HASH
-- If LAST_HASH IS NULL → NEW_RECORD (first time seen)
```

Pattern B — Snowflake Stream-Based CDC

Snowflake Streams attach to a table and track every DML operation since the last consumption. The stream maintains an OFFSET — a pointer to the last consumed position. Only rows changed since the last offset are visible when you query the stream.

Stream Metadata Column	Value	Meaning
METADATA\$ACTION	INSERT	Row was inserted (or the NEW value of an UPDATE)
METADATA\$ACTION	DELETE	Row was deleted (or the OLD value of an UPDATE)
METADATA\$ISUPDATE	TRUE	This INSERT/DELETE pair represents an UPDATE (not a net new/delete)
METADATA\$ISUPDATE	FALSE	Pure insert or pure delete
METADATA\$ROW_ID	varchar	Snowflake internal unique row ID — stable across updates

Pattern B: Stream-Based CDC

```
-- How UPDATES appear in Snowflake streams:
-- An UPDATE to customer C001's email generates TWO stream rows:
-- Row 1: METADATA$ACTION='DELETE', METADATA$ISUPDATE=TRUE ← old value
-- Row 2: METADATA$ACTION='INSERT', METADATA$ISUPDATE=TRUE ← new value

-- Query stream to see what changed
SELECT
    CUSTOMER_ID,
    EMAIL,
    METADATA$ACTION      AS ACTION,      -- INSERT or DELETE
    METADATA$ISUPDATE    AS IS_UPDATE,  -- TRUE if part of an update pair
    METADATA$ROW_ID      AS ROW_ID
FROM CDC_DB.STREAMS.STR_CUSTOMERS;

-- Correct pattern to consume only the NEW values of updates:
SELECT * FROM CDC_DB.STREAMS.STR_CUSTOMERS
WHERE METADATA$ACTION = 'INSERT'      -- take the new value
    AND METADATA$ISUPDATE = TRUE;      -- this is an update, not a net-new insert

-- Check if stream has unprocessed data (returns TRUE/FALSE)
SELECT SYSTEM$STREAM_HAS_DATA('CDC_DB.STREAMS.STR_CUSTOMERS');

-- CRITICAL: offset advances ONLY when DML consuming the stream COMMITS
-- If your task fails mid-run: offset does NOT advance → data preserved → safe to retry
```

Pattern C — Timestamp Watermark CDC

Use a watermark table to track 'last processed timestamp'. On each run, load rows where `_LOAD_TS > last` watermark. Simple but misses deletes — use with hash comparison for completeness.

Pattern C: Watermark CDC

```
-- Watermark-based incremental load
SELECT LAST_RUN_AT FROM CDC_DB.CONTROL.PIPELINE_WATERMARK
WHERE PIPELINE_NAME='CUSTOMER_CDC' AND STEP_NAME='BRONZE_TO_SILVER';

-- Load only new Bronze rows since last run
INSERT INTO CDC_DB.STAGING.STG_CUSTOMER_CHANGES
SELECT * FROM BRONZE_DB.RAW.RAW_CUSTOMERS
WHERE _LOAD_TS > :last_run_watermark;
```

CHAPTER 2

All SCD Types — Theory & Implementation

Slowly Changing Dimensions define HOW dimension attribute changes are handled in the data warehouse. Each type makes a different trade-off between storage cost, analytical flexibility, and implementation complexity.

2.1 SCD Type Comparison

Type	Name	On Change	History?	Storage	Example Use
Type 0	Fixed	Never change — original value kept forever	N/A	Minimal	DIM_DATE — 2024-01-15 is always a Monday
Type 1	Overwrite	Update in place — old value LOST	No	Minimal	DIM_STORES — store address updated
Type 2	Add Row	Expire old row, insert new row	Full	High	DIM_CUSTOMERS — segment change
Type 3	Add Column	Add prev_value column	Partial (1 prev only)	Low	PREV_SEGMENT on DIM_CUSTOMERS
Type 4	History Table	Separate audit/history table	Full	High	DIM_PRODUCT_PRICE_HISTORY
Type 6	Hybrid 1+2+3	SCD2 versioning + SCD1 overwrite + SCD3 prev col	Full + current	Very High	DIM_CUSTOMERS in this project

2.2 SCD Type 0 — Fixed / Retain Original

The original value is NEVER changed. Even if the business redefines a concept, the historical value is preserved. DIM_DATE is the canonical Type 0 example — 2024-01-15 is a Monday now and will always be a Monday.

SCD Type 0 — Fixed

```
-- SCD Type 0: no UPDATE statement ever runs on this table
-- The INSERT populates it once; no merge, no overwrite, no versioning.
CREATE TABLE DIM_DATE (
  DATE_KEY      NUMBER NOT NULL,  -- YYYYMMDD — immutable once loaded
  FULL_DATE     DATE NOT NULL,
  DAY_NAME      VARCHAR(10),      -- 'Monday' — never changes
  FISCAL_YEAR    NUMBER(4),        -- loaded once, never updated
  ...
  CONSTRAINT PK_DIM_DATE PRIMARY KEY (DATE_KEY)
);
-- Loaded ONCE. Never touched again.
-- If company changes fiscal year start: add NEW columns for new fiscal year
-- but NEVER update the existing FISCAL_YEAR column (that would rewrite history)
```

2.3 SCD Type 1 — Overwrite

UPDATE the existing row when an attribute changes. Old value is permanently lost. Use when: history has no analytical value, the change is a correction, or the attribute has too many changes to version.

SCD Type 1 — Overwrite

```
-- SCD Type 1 pattern: MERGE with UPDATE SET all columns
MERGE INTO SILVER_DB.DIMENSIONS.DIM_STORES tgt
USING (
  SELECT STORE_ID, STORE_NAME, CITY, STATE, MANAGER_ID, IS_ACTIVE, _LOAD_FILE
  FROM CDC_DB.STREAMS.STR_STORES
  WHERE METADATA$ACTION = 'INSERT'
  QUALIFY ROW_NUMBER() OVER (PARTITION BY STORE_ID ORDER BY _LOAD_TS DESC) = 1
) src
ON tgt.STORE_ID = src.STORE_ID
WHEN MATCHED THEN UPDATE SET      -- ALL columns overwritten, no history kept
  tgt.STORE_NAME   = src.STORE_NAME,
  tgt.CITY         = src.CITY,
  tgt.STATE        = src.STATE,
  tgt.MANAGER_ID   = src.MANAGER_ID,
  tgt.IS_ACTIVE    = src.IS_ACTIVE,
  tgt.DW_UPDATED_AT = CURRENT_TIMESTAMP()
WHEN NOT MATCHED THEN INSERT (STORE_ID, STORE_NAME, CITY, STATE, MANAGER_ID, IS_ACTIVE)
VALUES (src.STORE_ID, src.STORE_NAME, src.CITY, src.STATE, src.MANAGER_ID, src.IS_ACTIVE);

-- SCD1 result: always exactly ONE row per STORE_ID
-- History: NONE — previous manager_id is gone forever
```

2.4 SCD Type 2 — Add New Row (Full History)

The most important and widely implemented SCD type. When a tracked attribute changes: mark the existing row as expired, then insert a new 'current' row with the updated values. Every change is preserved as a separate row, enabling complete historical analysis.

SCD2 Column	Purpose	Example Values
EFF_START_DATE	When this version became effective	2023-01-15, 2024-03-20
EFF_END_DATE	When this version was superseded (NULL=current)	2024-03-19, NULL
IS_CURRENT	Quick filter for latest version	TRUE (current), FALSE (historical)
VERSION_NUM	Sequential version counter per business key	1, 2, 3...
SK_* (surrogate key)	Unique key for each version — used in fact FK	1001, 1002, 1003

SCD Type 2 — Three-Step Pattern

```
-- SCD Type 2: The three-step expire-and-insert pattern

-- ■■■ STEP 1: EXPIRE — mark current row as historical ■■■■■■■■■■
UPDATE DIM_CUSTOMERS tgt
SET
  EFF_END_DATE = CURRENT_DATE() - 1,  -- ended yesterday
  IS_CURRENT   = FALSE,
  DW_UPDATED_AT = CURRENT_TIMESTAMP()
```

2.5 SCD Type 3 — Add Previous Value Column

Add a PREV_VALUE column to the dimension to store one previous version of an attribute. Cheaper than SCD2 but only keeps one historical value. Best used as a companion to SCD2 when 'what was it before?' is a common query.

SCD Type 3 — Previous Value Column

```
-- SCD Type 3: column-level history
-- DIM_CUSTOMERS has both current AND previous segment columns:

SELECT
    CUSTOMER_ID,
    SEGMENT,           -- current segment (today)
    PREV_SEGMENT,      -- Type 3: what was it BEFORE the last change?
    LOYALTY_TIER,
    PREV_LOYALTY_TIER, -- Type 3: previous loyalty
    VERSION_NUM        -- how many times it changed (from SCD2)
FROM DIM_CUSTOMERS
WHERE CUSTOMER_ID = 'C001' AND IS_CURRENT = TRUE;

-- Result:
-- CUSTOMER_ID | SEGMENT      | PREV_SEGMENT | LOYALTY_TIER | PREV_LOYALTY_TIER
-- C001         | Corporate    | Consumer     | Platinum      | Gold
--
-- LIMITATION: only 1 previous value stored. If they change 3 times, you lose version 1.
-- For full history: use SCD Type 2 and query historical rows.
```

2.6 SCD Type 4 — History Table

Maintain a separate history table that stores ALL versions, while the main dimension table always contains only the current value. Gives fast current lookups + full audit trail.

SCD Type 4 — History Table

```
-- SCD Type 4: two-table approach
-- Main table: DIM_PRODUCTS (always current – fast joins)
-- History table: DIM_PRODUCT_PRICE_HISTORY (all price versions)

-- Query current price (fast – no filter needed)
SELECT PRODUCT_ID, PRODUCT_NAME, UNIT_PRICE
FROM DIM_PRODUCTS
WHERE IS_ACTIVE = TRUE; -- no IS_CURRENT needed! all rows are current

-- Query full price history (use history table)
SELECT
    h.PRODUCT_ID,
    h.UNIT_PRICE,
    h.VALID_FROM,
    h.VALID_TO,
    h.PRICE_CHANGE_PCT,
    CASE WHEN LAG(h.UNIT_PRICE) OVER (PARTITION BY h.PRODUCT_ID ORDER BY h.VALID_FROM)
         < h.UNIT_PRICE THEN 'Increase'
         WHEN LAG(h.UNIT_PRICE) OVER (PARTITION BY h.PRODUCT_ID ORDER BY h.VALID_FROM)
         > h.UNIT_PRICE THEN 'Decrease'
         ELSE 'Initial' END AS PRICE_DIRECTION
FROM DIM_PRODUCT_PRICE_HISTORY h
WHERE PRODUCT_ID = 'P001'
ORDER BY VALID_FROM;
```

2.7 SCD Type 6 — Hybrid (1+2+3)

Combines all three approaches: SCD2 versioning for full history, SCD1 overwrite for non-historical attributes, and SCD3 previous-value columns for quick before/after queries. DIM_CUSTOMERS in this project is a Type 6 implementation.

SCD Type 6 — Hybrid Implementation

```
-- SCD Type 6 in DIM_CUSTOMERS:
-- TYPE 2 columns (create new row on change):
--   SEGMENT, LOYALTY_TIER, PREFERRED_CHANNEL → each change = new version row
-- TYPE 1 columns (overwrite current row in place, no new version):
--   EMAIL, PHONE, CITY, STATE, CREDIT_LIMIT → latest value overwrites
-- TYPE 3 columns (keep previous value alongside current):
--   PREV_SEGMENT, PREV_LOYALTY_TIER → store one previous value

-- A single customer can have this profile after 3 updates:

-- VERSION 1: SEGMENT=Consumer, LOYALTY=Bronze,   EFF_START=2022-01-01, IS_CURRENT=FALSE
-- VERSION 2: SEGMENT=Consumer, LOYALTY=Gold,     EFF_START=2023-06-01, IS_CURRENT=FALSE
-- VERSION 3: SEGMENT=Corporate,LOYALTY=Platinum, EFF_START=2024-01-01, IS_CURRENT=TRUE
--           PREV_SEGMENT=Consumer, PREV_LOYALTY=Gold ← Type 3 on version 3

-- EMAIL column on VERSION 3 is ALWAYS the latest email (Type 1 overwrite applied to current row)
-- You don't have to query version 1 or 2 to find current email.

-- Query Type 6 dimension correctly:
SELECT
  CUSTOMER_ID,
  -- Type 1: always latest value
  EMAIL, PHONE, CITY,
  -- Type 2: value AT THIS VERSION
  SEGMENT, LOYALTY_TIER,
  -- Type 3: previous value
  PREV_SEGMENT,
  -- SCD2 versioning
  VERSION_NUM, IS_CURRENT, EFF_START_DATE, EFF_END_DATE
FROM DIM_CUSTOMERS
WHERE CUSTOMER_ID = 'C001'
ORDER BY VERSION_NUM;
```

CDC Classification Engine

3.1 CDC Classification Logic

03_cdc_engine/01_cdc_streams_control.sql

Page 8

3.2 The CDC Control Tables

Table	Purpose	Key Columns
CDC_STATE	Stores last-known hash per business key per table	TABLE_NAME, BUSINESS_KEY, LAST_HASH, CURRENT_VERSION
PIPELINE_WATERMARK	Tracks last successful run timestamp per step	PIPELINE_NAME, STEP_NAME, LAST_RUN_AT, STATUS
CDC_EVENT_LOG	Full audit trail of every change event	EVENT_TS, OPERATION, SCD_ACTION, BEFORE_HASH, AFTER_HASH, CHANGED_COLUMNS

KEY The CDC_EVENT_LOG is your forensic trail. Every column that changed, the hash before and after, and the SCD action taken.

CHAPTER 4

Silver Layer — All SCD Implementations

Full SQL implementation of every SCD type across five dimension tables.

4.1 Point-in-Time FK Resolution — The Most Critical SCD2 Concept

When loading a fact table that joins to an SCD2 dimension, you must find the dimension version that was ACTIVE when the business event occurred — not today's current version. This is called point-in-time (PIT) lookup.

CRITICAL WRONG: JOIN dim_customers c ON c.customer_id = f.customer_id AND c.is_current = TRUE → gives today's segment

TIP RIGHT: JOIN dim_customers c ON c.customer_id = f.customer_id AND order_date BETWEEN c.eff_start_date AND COALESCE(c.eff_end_date, '9999-12-31')

Point-in-Time FK Resolution

```
-- WRONG: always gives current segment, ignores history
SELECT f.ORDER_ID, f.TOTAL_AMOUNT, c.SEGMENT
FROM FACT_ORDERS f
JOIN DIM_CUSTOMERS c
  ON c.CUSTOMER_ID = f.CUSTOMER_ID
  AND c.IS_CURRENT = TRUE    -- ← WRONG: if customer changed segment since order, wrong segment!

-- RIGHT: find the version active when the order was placed
SELECT f.ORDER_ID, f.TOTAL_AMOUNT, c.SEGMENT
FROM FACT_ORDERS f
JOIN DIM_DATE d ON d.DATE_KEY = f.ORDER_DATE_KEY
JOIN DIM_CUSTOMERS c
  ON c.CUSTOMER_ID = f.CUSTOMER_ID
  AND d.FULL_DATE BETWEEN c.EFF_START_DATE
                        AND COALESCE(c.EFF_END_DATE, '9999-12-31')
-- This returns the segment the customer was in WHEN they placed the order
-- even if they've since moved to a different segment

-- Stored in FACT_ORDERS: SK_CUSTOMER is the surrogate key of the version active at order time
-- So this also works:
SELECT f.ORDER_ID, c.SEGMENT
FROM FACT_ORDERS f
JOIN DIM_CUSTOMERS c ON c.SK_CUSTOMER = f.SK_CUSTOMER
-- SK_CUSTOMER was set at load time using the PIT lookup above
-- Future segment changes generate new SK values – the old SK is never reused
```

04_silver_scd/01_silver_all_scd_types.sql

```
-- =====
-- FILE: 04_silver_scd/01_silver_all_scd_types.sql
-- PURPOSE: All four SCD types – tables, procedures, examples
--
-- SCD TYPE 0 – Fixed / Retain Original (DIM_DATE, DIM_REGIONS)
-- SCD TYPE 1 – Overwrite (DIM_STORES – latest value, no history)
-- SCD TYPE 2 – Add Row (DIM_CUSTOMERS, DIM_PRODUCTS, DIM_EMPLOYEES)
-- SCD TYPE 3 – Add Column (DIM_CUSTOMERS – previous segment)
-- SCD TYPE 4 – History Table (DIM_PRODUCT_PRICE_HISTORY)
```

CHAPTER 5

Gold Layer & Complete Task DAG Pipeline

Gold aggregations leverage SCD2 history for segment-based analysis. The 10-task DAG runs hourly and processes all CDC changes end-to-end.

5.1 Why SCD2 Matters in Gold Analytics

Without SCD2 point-in-time resolution, segment analysis is corrupted. Consider a customer who was 'Consumer' for 2 years then switched to 'Corporate'. Their 2 years of Consumer orders would incorrectly show as Corporate orders if you only ever join to IS_CURRENT=TRUE.

SCD2 Analytics — Segment Revenue

```
-- SCD2-powered segment revenue analysis
-- Revenue is attributed to the segment the customer was in AT TIME OF PURCHASE
SELECT
    d.YEAR,
    d.QUARTER_NAME,
    c.SEGMENT,          -- segment at ORDER TIME (from SCD2 FK in fact)
    COUNT(DISTINCT f.ORDER_ID) AS ORDERS,
    SUM(f.NET_AMOUNT) AS REVENUE,
    -- YTD cumulative by segment
    SUM(SUM(f.NET_AMOUNT)) OVER (
        PARTITION BY d.YEAR, c.SEGMENT
        ORDER BY d.MONTH_NUM
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS YTD_REVENUE
FROM SILVER_DB.FACTS.FACT_ORDERS f
JOIN SILVER_DB.DIMENSIONS.DIM_DATE d ON d.DATE_KEY = f.ORDER_DATE_KEY
JOIN SILVER_DB.DIMENSIONS.DIM_CUSTOMERS c ON c.SK_CUSTOMER = f.SK_CUSTOMER
-- c.SEGMENT is the segment at time of order – correct SCD2 usage!
GROUP BY 1,2,3
ORDER BY 1,2,3;

-- Customer segment journey view – shows all transitions
SELECT
    CUSTOMER_ID,
    VERSION_NUM,
    SEGMENT_AT_VERSION,
    PREV_SEGMENT,
    EFF_START_DATE,
    EFF_END_DATE,
    DAYS_AT_VERSION,
    REVENUE_IN_THIS_VERSION -- what they spent WHILE in this segment
FROM GOLD_DB.REPORTING.V_CUSTOMER_SEGMENT_JOURNEY
WHERE CUSTOMER_ID = 'C001';
```

5.2 Task DAG Structure

Task	Type	Dependencies	Purpose
------	------	--------------	---------

TASK_CDC_PIPELINE_ROOT	Root (scheduler)	None — CRON hourly	Entry point — only runs when streams have data
TASK_CLASSIFY_CUSTOMERS	Child (parallel)	AFTER ROOT	Run CDC classification for customers
TASK_CLASSIFY_PRODUCTS	Child (parallel)	AFTER ROOT	Run CDC classification for products
TASK_CLASSIFY_EMPLOYEES	Child (parallel)	AFTER ROOT	Run CDC classification for employees
TASK_SCD1_STORES	Child (parallel)	AFTER ROOT	SCD1 upsert for stores
TASK_SCD2_CUSTOMERS	Child	AFTER CLASSIFY_CUSTOMERS + CLASSIFY_EMPLOYEES	Full SCD2 customer dimension load
TASK_SCD2_PRODUCTS	Child	AFTER CLASSIFY_PRODUCTS	Full SCD2 product dimension load
TASK_LOAD_FACTS	Child	AFTER all SCD2 dims	Fact load with PIT FK resolution
TASK_BUILD_DAILY_SUMMARY	Child (parallel)	AFTER FACTS	Gold daily aggregation
TASK_BUILD_CUSTOMER_360	Child (parallel)	AFTER FACTS	Gold customer 360 + RFM
TASK_PIPELINE_COMPLETE	Leaf	AFTER both Gold tasks	Log success, update watermark

CRITICAL Resume order: LEAF tasks first, ROOT task LAST. If you resume ROOT before TASK_LOAD_FACTS is resumed, the fir

05_gold/01_gold_and_pipeline.sql

```
-- =====
-- FILE: 05_gold/01_gold_and_pipeline.sql
-- PURPOSE: Gold aggregations + complete Task DAG pipeline
-- =====

USE ROLE DATA_ENG_ROLE;
USE WAREHOUSE TRANSFORM_WH;
USE DATABASE GOLD_DB;

-- =====
-- GOLD TABLES
-- =====

USE SCHEMA SALES_MART;

CREATE TABLE IF NOT EXISTS DAILY_SALES_SUMMARY (
    SUMMARY_KEY          NUMBER AUTOINCREMENT PRIMARY KEY,
    ORDER_DATE_KEY       NUMBER,
    FULL_DATE            DATE,
    REGION_ID            VARCHAR(20),
    SEGMENT              VARCHAR(30),      -- from SCD2 dim at order time
    ORDER_COUNT          NUMBER,
    UNIQUE_CUSTOMERS     NUMBER,
    GROSS_REVENUE         NUMBER(14,2),
    NET_REVENUE          NUMBER(14,2),
    TOTAL_PROFIT         NUMBER(14,2),
    AVG_ORDER_VALUE      NUMBER(10,2),
```

CHAPTER 6

Security, Monitoring & Testing

Masking policies protect PII, row access policies enforce segment-level access, and a comprehensive test suite validates CDC and SCD2 correctness.

6.1 Security Layers

Control	Applied To	What It Does
MASK_EMAIL	DIM_CUSTOMERS.EMAIL, DIM_EMPLOYEES.EMAIL	Engineers see full email; senior analysts see partial; others see ***@***.***
MASK_SALARY	DIM_EMPLOYEES.SALARY	Engineers see full; senior analysts see rounded to nearest 1000; others see NULL
POLICY_SEGMENT (row)	GOLD daily summary	Users only see rows for their assigned segments — from USER_SEGMENT_ACCESS mapping table
TAG_PII	EMAIL, SALARY, PHONE, CUSTOMER_NAME	Governance tags for compliance reporting and masking policy discovery
NP_CORPORATE	Account-level	Restrict all connections to corporate IPs and VPN
NP_ETL_SERVERS	ETL service account	Restrict ETL service account to data center IPs only

6.2 CDC & SCD2 Test Scenarios

09_testing/01_cdc_scd_tests.sql

```

LSE 0 END) AS MISSING_EMP_FK,
    ROUND(SUM(CASE WHEN SK_CUSTOMER IS NULL THEN 1 ELSE 0 END)*100.0/COUNT(*),2) AS CUST_MISS_PCT
FROM SILVER_DB.FACTS.FACT_ORDERS
WHERE DW_CREATED_AT >= DATEADD('day',-1,CURRENT_TIMESTAMP());

-- Warehouse credit consumption
SELECT WAREHOUSE_NAME, SUM(CREDITS_USED) AS CREDITS, COUNT(*) AS SESSIONS
FROM SNOWFLAKE.ACCOUNT_USAGE.WAREHOUSE_METERING_HISTORY
WHERE START_TIME >= DATE_TRUNC('month',CURRENT_TIMESTAMP())
GROUP BY 1 ORDER BY 2 DESC;

-- =====
-- FILE: 09_testing/01_cdc_scd_tests.sql
-- Test scenarios for CDC and SCD2
-- =====

-- TEST 1: Simulate customer segment change (SCD2 trigger)
-- This should create a new DIM_CUSTOMERS row with higher VERSION_NUM
INSERT INTO BRONZE_DB.RAW.RAW_CUSTOMERS
    (CUSTOMER_ID, CUSTOMER_NAME, EMAIL, SEGMENT, LOYALTY_TIER, PREFERRED_CHANNEL,
     CITY, STATE, COUNTRY, REGION_ID, IS_ACTIVE, _LOAD_BATCH_ID, _SRC_ROW_HASH)
VALUES
    ('C001','Alice Johnson','alice@example.com','Corporate','Gold','Email',
     'New York','NY','USA','R1','TRUE',
     'TEST_BATCH_001',
     MD5('C001|Alice Johnson|alice@example.com|Corporate|Gold|Email|New York|NY|USA|R1|TRUE'));

```

6.3 Key Monitoring Queries

Monitoring Queries

```
-- CDC event summary (last 24h)
SELECT CDC_OPERATION, SCD_ACTION, COUNT(*) AS EVENTS, COUNT(DISTINCT BUSINESS_KEY) AS KEYS
FROM CDC_DB.CONTROL.CDC_EVENT_LOG
WHERE EVENT_TS >= DATEADD('hour', -24, CURRENT_TIMESTAMP())
GROUP BY 1,2 ORDER BY 3 DESC;

-- SCD2 version depth – how versioned is each dimension?
SELECT 'DIM_CUSTOMERS' AS DIM,
       MAX(VERSION_NUM) AS MAX_VERSIONS,
       AVG(VERSION_NUM) AS AVG_VERSIONS,
       COUNT(DISTINCT CUSTOMER_ID) AS UNIQUE_KEYS,
       COUNT(*) AS TOTAL_ROWS
FROM SILVER_DB.DIMENSIONS.DIM_CUSTOMERS
UNION ALL
SELECT 'DIM_PRODUCTS', MAX(VERSION_NUM), AVG(VERSION_NUM),
       COUNT(DISTINCT PRODUCT_ID), COUNT(*)
FROM SILVER_DB.DIMENSIONS.DIM_PRODUCTS;

-- Fact FK coverage – are all facts linked to dimension versions?
SELECT
    COUNT(*) AS TOTAL_ORDERS,
    SUM(CASE WHEN SK_CUSTOMER IS NULL THEN 1 ELSE 0 END) AS ORPHANED_CUSTOMER,
    SUM(CASE WHEN SK_EMPLOYEE IS NULL THEN 1 ELSE 0 END) AS ORPHANED_EMPLOYEE,
    ROUND(SUM(CASE WHEN SK_CUSTOMER IS NULL THEN 1 ELSE 0 END)*100.0/COUNT(*),2) AS ORPHAN_PCT
FROM SILVER_DB.FACTS.FACT_ORDERS
WHERE DW_CREATED_AT >= DATEADD('day', -1, CURRENT_TIMESTAMP());

-- Pipeline task run history
SELECT NAME, STATE, ERROR_MESSAGE,
       DATEDIFF('second', QUERY_START_TIME, COMPLETED_TIME) AS DURATION_SEC
FROM TABLE(INFORMATION_SCHEMA.TASK_HISTORY(
    SCHEDULED_TIME_RANGE_START => DATEADD('hour', -24, CURRENT_TIMESTAMP())
))
ORDER BY SCHEDULED_TIME DESC LIMIT 50;
```

6.4 SCD Type Quick Reference

Scenario	SCD Type	Pattern	Key SQL
Customer email changed (correction)	Type 1	Overwrite	MERGE → UPDATE all cols when matched
Customer segment: Consumer → Corporate	Type 2	Expire + Insert	UPDATE EFF_END, IS_CURRENT=F; INSERT new row
Customer moved city (correction)	Type 1 (in Type 6)	Overwrite current row	UPDATE current row, no new version
Product price changed	Type 2	Expire + Insert + Type4	New version + insert to price history table
Store renamed	Type 1	Overwrite	MERGE UPDATE
Employee promoted to Manager	Type 2	Expire + Insert	New version with new JOB_TITLE
Employee transferred to new dept	Type 2	Expire + Insert	Facts show old dept for old orders

Customer account deleted	Soft Delete (Type 1)	IS_ACTIVE=F ALSE	Never physically delete — update flag
New date added to DIM_DATE	Type 0	Append only	INSERT new date; never touch old dates

KEY The golden rule: NEVER physically DELETE from a dimension table. Always soft-delete (IS_ACTIVE=FALSE). Fact rows that